The Transformer — Complete Guide from Input to Output

Primary Source: Vaswani, A., Shazeer, N., Parmar, N., et al. “Attention Is All You Need.” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.

Model Configuration (from the original paper, Table 3): - `d_model` (model dimension) = 512 - `h` (number of attention heads) = 8 - `d_k` = `d_v` (key/value dimension per head) = 64 - `d_ff` (feed-forward inner dimension) = 2048 - `N` (number of encoder & decoder layers) = 6 - Vocabulary size = 37,000 (BPE tokens in the original WMT experiments)

The Big Picture — What Are We Building?

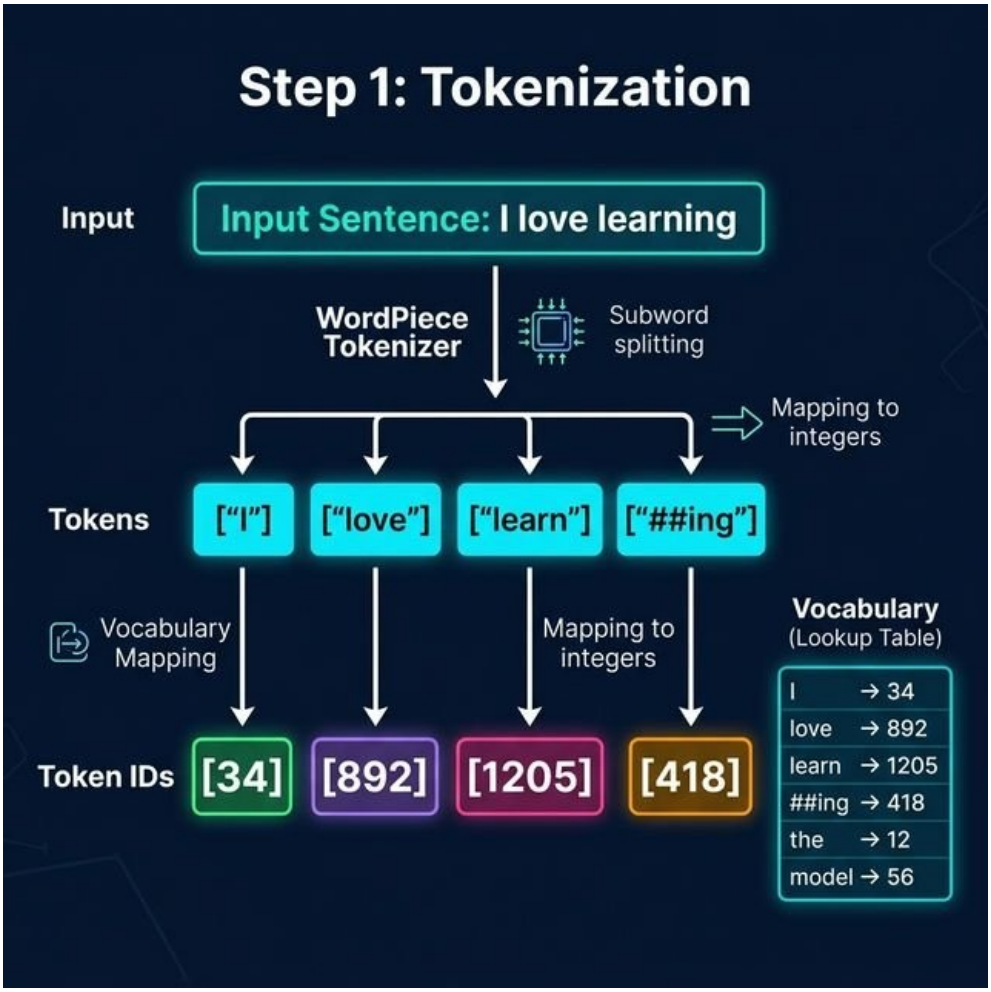
The Transformer takes a sequence of tokens (words/subwords) as input and produces a sequence of tokens as output. It was originally designed for **machine translation** (e.g., English → French) but is now the backbone of GPT, Claude, Gemini, BERT, and virtually every modern AI model.

```
"I love learning"  ──►  [TRANSFORMER]  ──►  "J'adore apprendre"
     (input)                                     (output)
```

The architecture has two halves: - **Encoder**: Reads the entire input at once and builds a rich understanding of it - **Decoder**: Generates the output one token at a time, attending to the encoder’s understanding

We will go through **every single step** from raw text input to final output token.

STEP 1: Tokenization (Breaking Text into Pieces)



Step 1: Tokenization — converting text into token IDs

What Happens Here

The raw text string is split into **tokens** (subword units) and each token is mapped to an integer **Token ID** using a fixed vocabulary.

```
Input text:     "I love learning"
                     ↓ Tokenizer
Tokens:         ["I", "love", "learn", "##ing"]
                     ↓ Vocabulary lookup
Token IDs:      [34, 892, 1205, 418]
```

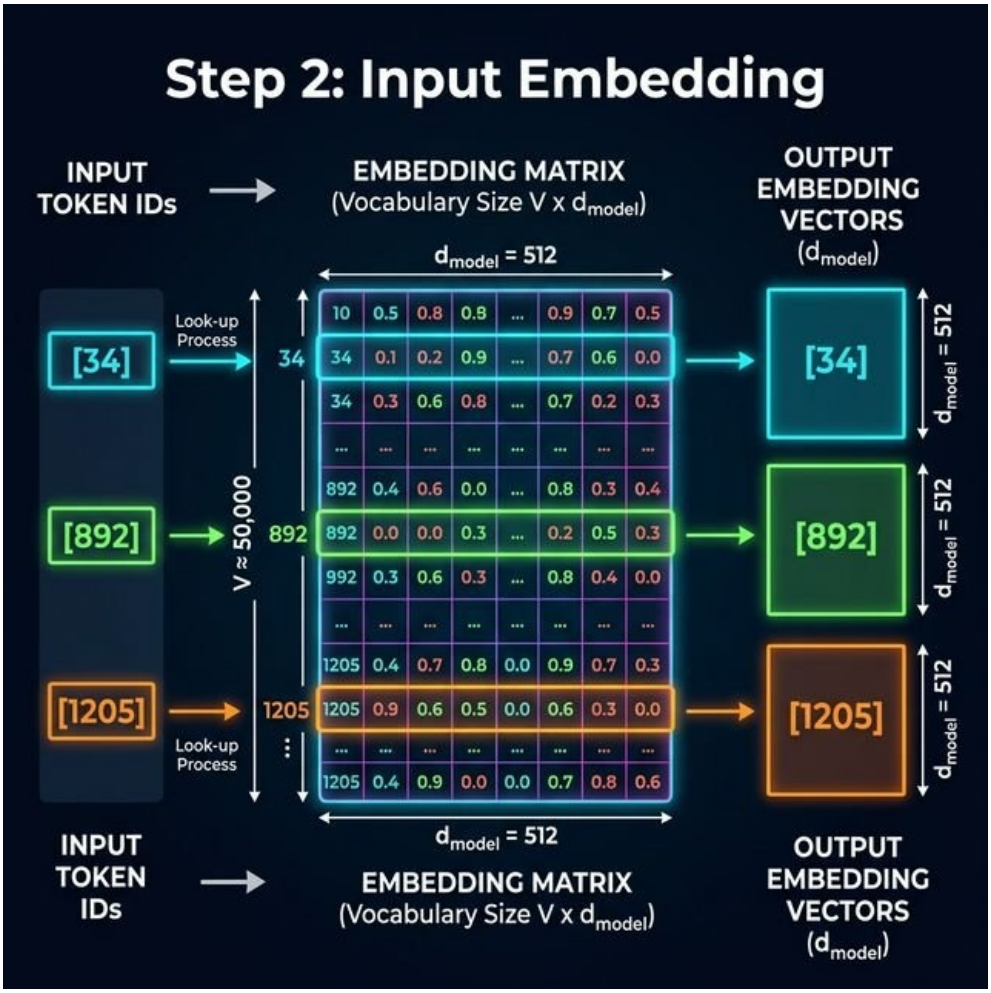
Why Subwords, Not Whole Words?

Approach	Example	Problem
Character-level	[“I”, ” “,”l”, “o”, “v”, “e”, ...]	Sequences become extremely long; hard to capture meaning
Word-level	[“I”, “love”, “learning”]	Vocabulary would need millions of entries; can’t handle unseen words like “ChatGPTing”
Subword (BPE)	[“I”, “love”, “learn”, “##ing”]	✅ Compact vocabulary (~30K-100K); handles any word by breaking it into known pieces

BPE (Byte Pair Encoding, published by Sennrich et al., 2016, arXiv:1508.07909) iteratively merges the most frequent character pairs in the training corpus until the vocabulary reaches the desired size.

> 🔍 **What if we skip tokenization?** The model would receive raw characters — sequences become 4-5× longer, training becomes much slower, and the model struggles to learn word-level meaning because each character carries very little semantic (meaning-related) information.

STEP 2: Input Embedding (Giving Tokens Meaning)



Step 2: Input Embedding — looking up dense vector representations

What Happens Here

Each Token ID is converted into a **dense vector** (a list of 512 numbers) by looking it up in a learned **Embedding Matrix**.

```
Token ID: 892 ("love")
                ↓ Look up row 892 in embedding matrix
Embedding:  [0.21, -0.45, 0.87, 0.03, ..., -0.12]   ← 512 numbers
```

The Math

```
Embedding Matrix E:  shape = (vocab_size × d_model) = (37000 × 512)

For token ID = 892:
    embedding_vector = E[892]     ← just pick row 892
```

```
For sequence [34, 892, 1205, 418]:
    X = E[[34, 892, 1205, 418]]  ← shape becomes (4 × 512)
```

The embedding matrix `E` is a **learned parameter** — it starts random and gets updated during training via backpropagation (the process where gradients flow backward through the network to adjust weights).

Why Not One-Hot Encoding?

Approach	Vector for "love" (vocab=37K)	Dimension	Problem
One-hot	[0, 0, ..., 1, ..., 0, 0]	37,000	Enormous, sparse (mostly zeros), no relationships between words
Embedding	[0.21, -0.45, 0.87, ...]	512	Compact, dense, and **similar words end up near each other** in this space

> 🔍 **What if we use one-hot instead of embeddings?** - Vectors become 37K-dimensional instead of 512 — **72× larger** - "love" and "adore" would be equally distant as "love" and "table" — **no semantic relationships** - Memory usage explodes; training becomes impractical at scale

Important Scaling Factor

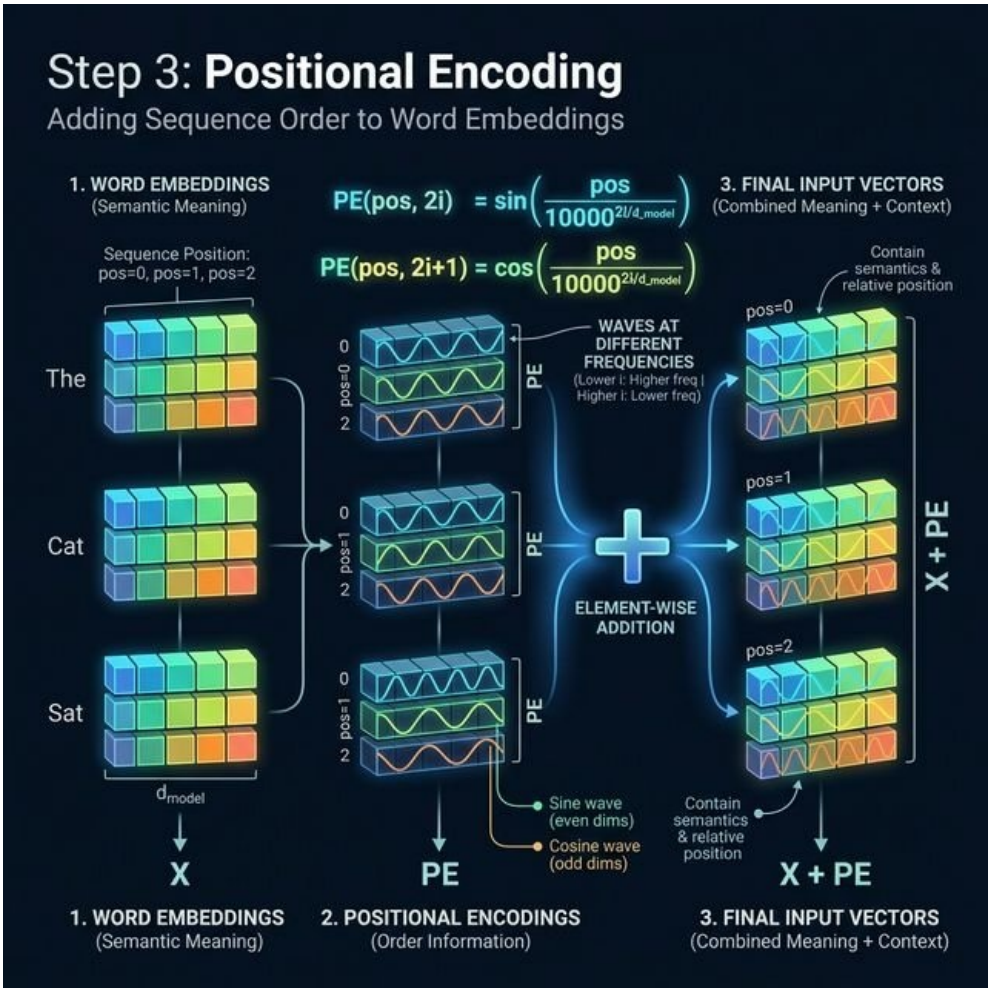
The original paper **multiplies the embedding by √d_model** (√512 ≈ 22.6):

```
X = E[token_ids] × √d_model
```

Why? Because embeddings are initialized with small values (variance ≈ 1/d_model). Without scaling, the positional encoding (added next) would dominate the signal. Multiplying by √d_model brings the embedding magnitude closer to the positional encoding magnitude, ensuring both signals contribute equally.

> 🔍 **What if we skip the √d_model scaling?** The positional encoding signal overwhelms the semantic (word meaning) signal. The model would know WHERE each token is but forget WHAT each token means.

STEP 3: Positional Encoding (Teaching the Model Word Order)



Step 3: Positional Encoding — adding position information via sine/cosine waves

The Problem

Unlike RNNs that process words one-by-one (so position is implicit), the Transformer processes all words **simultaneously**. Without explicit position information:

```
"Dog bites man"  and  "Man bites dog"
would look IDENTICAL to the model — same words, same embeddings, different meaning!
```

The Solution

Add a **Positional Encoding** vector to each embedding. The original paper uses fixed sinusoidal (sine and cosine wave-based) functions:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))

Where:
  pos   = position in the sequence (0, 1, 2, 3, ...)
  i     = dimension index (0, 1, 2, ..., 255)
  d_model = 512
```

Concrete Example

```
For position 0, dimension 0:
  PE(0, 0) = sin(0 / 10000^(0/512)) = sin(0) = 0.000
```

```
For position 1, dimension 0:
  PE(1, 0) = sin(1 / 10000^(0/512)) = sin(1) = 0.841

For position 1, dimension 1:
  PE(1, 1) = cos(1 / 10000^(0/512)) = cos(1) = 0.540

For position 1, dimension 100:
  PE(1, 100) = sin(1 / 10000^(100/512)) = sin(1/10000^0.195) = sin(0.402) = 0.391
```

Then We ADD Them

```
Final_Input = Word_Embedding + Positional_Encoding

  [0.21, -0.45, 0.87, ...]    ← what the word means
+  [0.00,  1.00, 0.84, ...]    ← where the word is
= [0.21,  0.55, 1.71, ...]    ← meaning + position combined
```

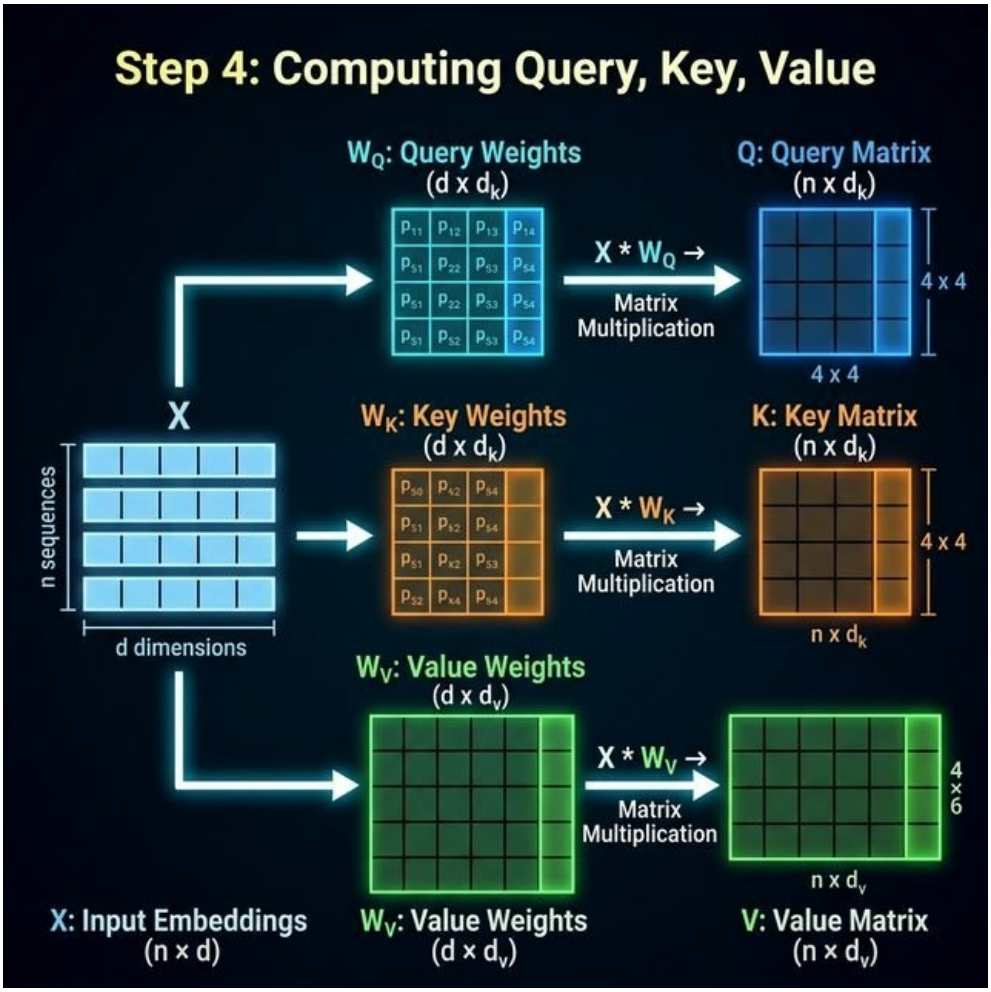
Why Sinusoidal? Why Not Learned Positions?

Approach	Description	Pros	Cons
Sinusoidal (original)	Fixed math function, not learned	Can generalize to longer sequences than seen in training; no extra parameters	Slightly less optimal on any fixed length
Learned	A separate embedding matrix for positions	Slightly better on training-length sequences	Cannot generalize beyond training length; adds parameters
RoPE (Su et al., 2021)	Rotates Q/K vectors based on position	Best for long contexts; used in GPT-4, Claude, LLaMA	More complex mathematics

The original Transformer paper found “nearly identical results” between sinusoidal and learned (Section 3.5, Table 3, Row (E)).

> 🔍 **What if we remove positional encoding entirely?** The model becomes a **bag-of-words** (unordered set of words) processor. “I ate the fish” and “the fish ate I” become identical. Word order is completely lost. Performance drops catastrophically on any order-sensitive task.

STEP 4: Computing Query, Key, Value (The Foundation of Attention)



Step 4: Computing Q, K, V from the input via learned weight matrices

The Intuition — A Library Analogy

Imagine you're in a library: - **Query (Q)**: Your question — "I need a book about deep learning" - **Key (K)**: The label on each book spine — "Machine Learning", "Deep Learning", "Cooking" - **Value (V)**: The actual content inside each book

You **compare your Query to every Key** to find the most relevant books, then **read the Values** of the best matches. That's exactly what attention does.

The Math

For input matrix `X` (shape: seq_len × d_model = 4 × 512):

```
Q = X · W_Q      (shape: 4 × 512  ·  512 × 64  =  4 × 64)
K = X · W_K      (shape: 4 × 512  ·  512 × 64  =  4 × 64)
V = X · W_V      (shape: 4 × 512  ·  512 × 64  =  4 × 64)

Where W_Q, W_K, W_V are LEARNED weight matrices (512 × 64 each)
```

Each token position gets its own Q, K, and V vector. `d_k = 64` for a single head (since d_model / h = 512 / 8 = 64).

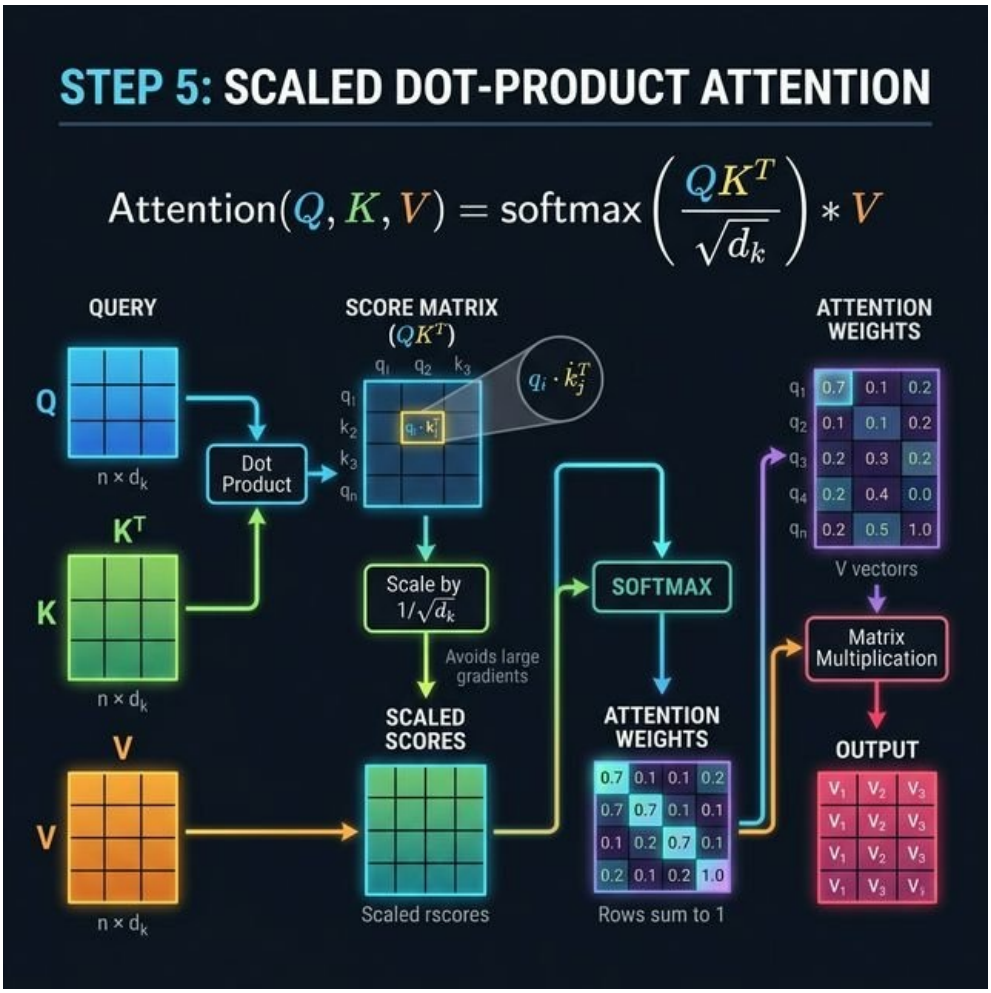
Why Three SEPARATE Projections?

> 🔍 **What if we use the same matrix for Q, K, and V (Q = K = V = X)?** This is called **identity**

> **attention** — every token would attend to itself the most (since a vector's dot-product with itself is maximum). The model loses the ability to learn asymmetric relationships (e.g., "pronouns should attend to nouns, but not vice versa").

> 🔍 **What if we only use Q and K (no V)?** We compute *which* tokens are relevant, but we have no mechanism to *extract* information from them. V is the actual information being gathered. Without V, attention scores exist but have nothing to aggregate.

STEP 5: Scaled Dot-Product Attention (The Core Mechanism)



Step 5: Scaled Dot-Product Attention — the heart of the Transformer

The Formula

```
Attention(Q, K, V) = softmax( Q · K^T / √d_k ) · V
```

Let's break this down step-by-step with a concrete 3-token example.

Step 5.1: Compute Raw Attention Scores (Q · K^T)

```
Q · K^T = matrix multiplication

     K₁^T    K₂^T    K₃^T
```

```
Q₁ [ 112     45     78  ]     ← How relevant is each key to query 1?
Q₂ [  34    156     23  ]     ← How relevant is each key to query 2?
Q₃ [  67     89    134  ]     ← How relevant is each key to query 3?

Shape: (3 × 64) · (64 × 3) = (3 × 3)
```

The dot product measures **similarity** between Q and K vectors. Higher dot product = more relevant.

Step 5.2: Scale by √d_k (Division by √64 = 8)

```
        K₁^T   K₂^T   K₃^T
Q₁ [   14.0    5.6    9.8  ]
Q₂ [    4.3   19.5    2.9  ]      ← values are now manageable
Q₃ [    8.4   11.1   16.8  ]
```

Why Scale? (Critical Design Decision)

Without scaling, when d_k is large (e.g., 64), the dot products `Q · K^T` produce very large values. When you feed large values into softmax:

```
softmax([100, 101, 102]) ≈ [0.09, 0.24, 0.67]   ← reasonable differences
softmax([1000, 1001, 1002]) ≈ [0.00, 0.00, 1.00]  ← essentially ONE-HOT!
```

> 🔍 **What if we don't scale (no division by √d_k)?** Softmax becomes **extremely peaked** (all probability mass on one token). Gradients become nearly zero for all other positions. The model can only attend to ONE token per position — it can't blend information from multiple relevant tokens. Training becomes unstable.

The paper states this explicitly: *"We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients."* (Section 3.2.1)

Step 5.3: Apply Softmax (Convert to Probabilities)

```
softmax is applied ROW-WISE (each row sums to 1.0):

Input:          [14.0,  5.6,  9.8]
                    ↓ softmax
Output:         [0.89, 0.01, 0.10]     ← "Token 1 mostly attends to itself"

Full matrix (Attention Weights):
        K₁      K₂      K₃
Q₁ [  0.89    0.01    0.10  ]    sum = 1.0
Q₂ [  0.02    0.95    0.03  ]    sum = 1.0
Q₃ [  0.05    0.12    0.83  ]    sum = 1.0
```

Softmax formula for a single row:

```
softmax(z_i) = e^(z_i) / Σ_j e^(z_j)

For [14.0, 5.6, 9.8]:
  e^14.0 = 1,202,604
  e^5.6  = 270
  e^9.8  = 18,034
  sum    = 1,220,908
```

```
softmax = [1202604/1220908, 270/1220908, 18034/1220908]
        = [0.985, 0.0002, 0.015]
```

Step 5.4: Multiply by V (Gather Information)

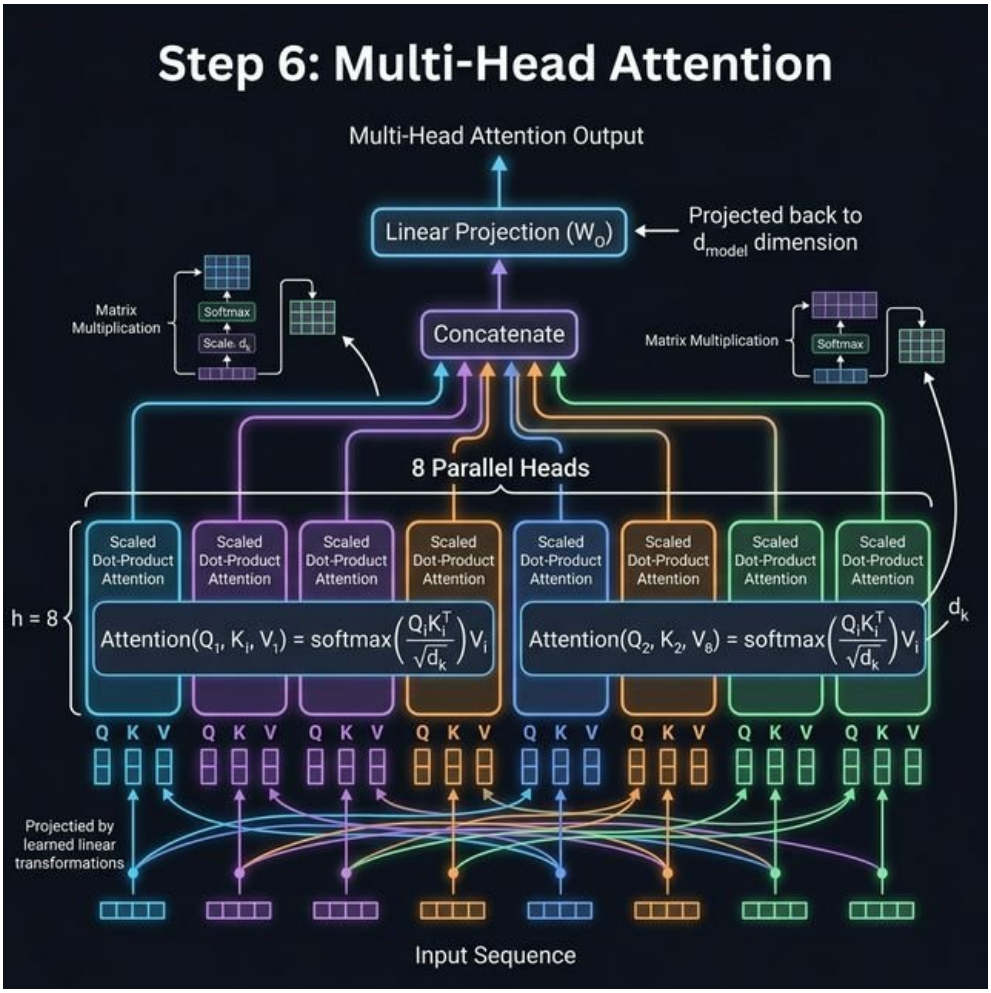
```
Attention_Output = Attention_Weights · V

[0.89  0.01  0.10]      [V₁]      [0.89·V₁ + 0.01·V₂ + 0.10·V₃]
[0.02  0.95  0.03]  ·   [V₂]  =   [0.02·V₁ + 0.95·V₂ + 0.03·V₃]
[0.05  0.12  0.83]      [V₃]      [0.05·V₁ + 0.12·V₂ + 0.83·V₃]

Each output row is a WEIGHTED AVERAGE of all value vectors,
where the weights tell us how much to "attend to" each position.
```

> 🔍 **What if we use softmax over columns instead of rows?** Each *Key* position would have attention weights summing to 1.0 — every key distributes its value content equally. But we want each *Query* to decide what to look at. Column-wise softmax answers the wrong question.

STEP 6: Multi-Head Attention (Learning Multiple Relationships)



Step 6: Multi-Head Attention — parallel attention heads capturing different patterns

The Problem with Single-Head Attention

One attention head can only learn **one type of relationship**. But language has MANY simultaneous relationships:

```
"The cat sat on the mat because it was soft"

Relationships to capture simultaneously:
  - Syntactic (grammar-related): "cat" → "sat" (subject-verb)
  - Coreference (pronoun resolution): "it" → "mat" (what is "it"?)
  - Spatial (location): "sat" → "on" → "mat" (where?)
  - Causal (reason): "because" → "soft" (why?)
```

The Solution: Multiple Parallel Heads

```
Instead of ONE attention with d_k = 512:
  → EIGHT heads, each with d_k = 64  (512 / 8 = 64)

Each head has its OWN W_Q, W_K, W_V matrices:
  Head 1: Q₁ = X·W_Q¹,  K₁ = X·W_K¹,  V₁ = X·W_V¹  → Attention₁
  Head 2: Q₂ = X·W_Q²,  K₂ = X·W_K²,  V₂ = X·W_V²  → Attention₂
  ...
  Head 8: Q₈ = X·W_Q⁸,  K₈ = X·W_K⁸,  V₈ = X·W_V⁸  → Attention₈
```

Concatenate + Project

```
MultiHead(Q, K, V) = Concat(head₁, head₂, ..., head₈) · W_O

Where:
  head_i = Attention(Q·W_Qⁱ, K·W_Kⁱ, V·W_Vⁱ)    shape: (seq_len × 64)
  Concat(...)                                        shape: (seq_len × 512)
  W_O                                                shape: (512 × 512)
  Final output                                       shape: (seq_len × 512)
```

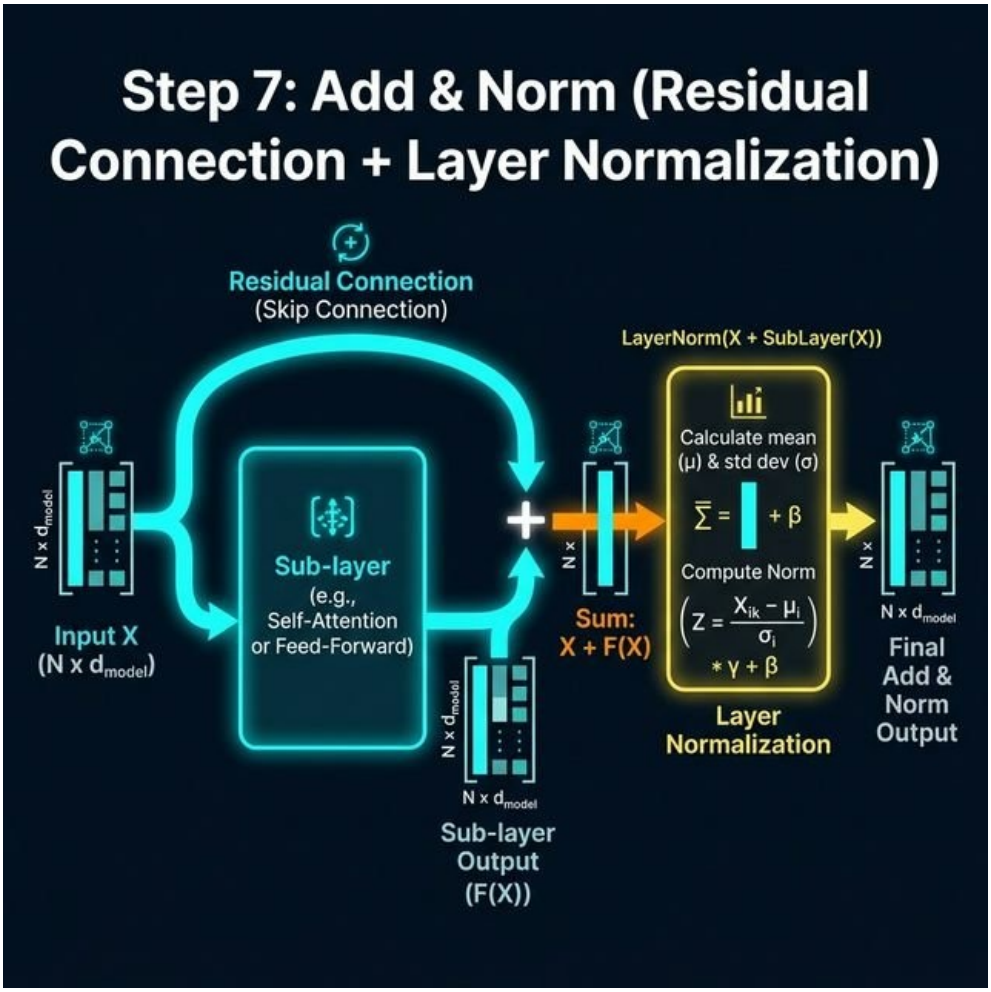
Parameter Count for Multi-Head Attention

```
Per head:   W_Qⁱ (512×64) + W_Kⁱ (512×64) + W_Vⁱ (512×64) = 98,304
All 8 heads: 98,304 × 8 = 786,432
Output projection W_O: 512 × 512 = 262,144
                                   ________
Total: 1,048,576 parameters (≈ 1M per attention layer)
```

> 🔍 **What if we use 1 head instead of 8?** The paper tested this (Table 3, Row B): performance dropped by 0.9 BLEU (translation quality metric). A single head captures only one relationship pattern. Multi-head gives the model **orthogonal (independent, non-overlapping) views** of the same data.

> 🔍 **What if we use 512 heads (d_k = 1)?** Each head has only 1 dimension — it can only compute a single scalar dot-product. Too little information per head for meaningful attention patterns.

STEP 7: Add & Norm (Residual Connections + Layer Normalization)



Step 7: Add & Norm — skip connections prevent information loss

What Happens Here

After each sub-layer (attention or feed-forward), two operations are applied:

```
Output = LayerNorm(X + SubLayer(X))
                   ______________
                   residual connection (skip connection)
```

7.1: Residual Connection (the "+" in Add & Norm)

```
Original input:    X               = [0.21, -0.45, 0.87, ...]
Attention output:  SubLayer(X)     = [0.05,  0.32, -0.11, ...]
                                       ↓ Add them
Residual output:   X + SubLayer(X) = [0.26, -0.13, 0.76, ...]
```

Why? (Published by He et al., 2015, "Deep Residual Learning," arXiv:1512.03385)

In deep networks (the Transformer has 6 layers × 2 sub-layers = 12 stacked components), gradients can vanish or explode as they flow backward. The skip connection creates a **direct path** for the gradient:

```
Without residual:   gradient must flow THROUGH every layer → degrades
With residual:      gradient can flow AROUND layers via skip connection → preserved

Mathematically:
  ∂Loss/∂X = ∂Loss/∂Output · (1 + ∂SubLayer/∂X)
                                 ─
```

```
                                This "1" is the direct skip path
                                Gradient is AT LEAST 1, never vanishes!
```

> 🔍 **What if we remove residual connections?** Training a 6-layer Transformer becomes very difficult — gradients vanish in deeper layers, early layers barely learn. The paper does not report this ablation (removal of a component to test its effect) explicitly, but it's a well-established result in deep learning (He et al., 2015).

7.2: Layer Normalization (the "Norm" in Add & Norm)

```
For a vector z = [z₁, z₂, ..., z_d]:

  μ = mean(z) = (1/d) · Σ z_i                    ← compute mean
  σ = sqrt((1/d) · Σ (z_i - μ)²)                 ← compute standard deviation

  LayerNorm(z) = γ · (z - μ) / (σ + ε) + β       ← normalize, then scale & shift

Where:
  γ (gamma) = learned scale parameter (initialized to 1)
  β (beta)  = learned shift parameter (initialized to 0)
  ε (epsilon) = tiny constant (e.g., 10⁻⁶) to prevent division by zero
```

Concrete Example:

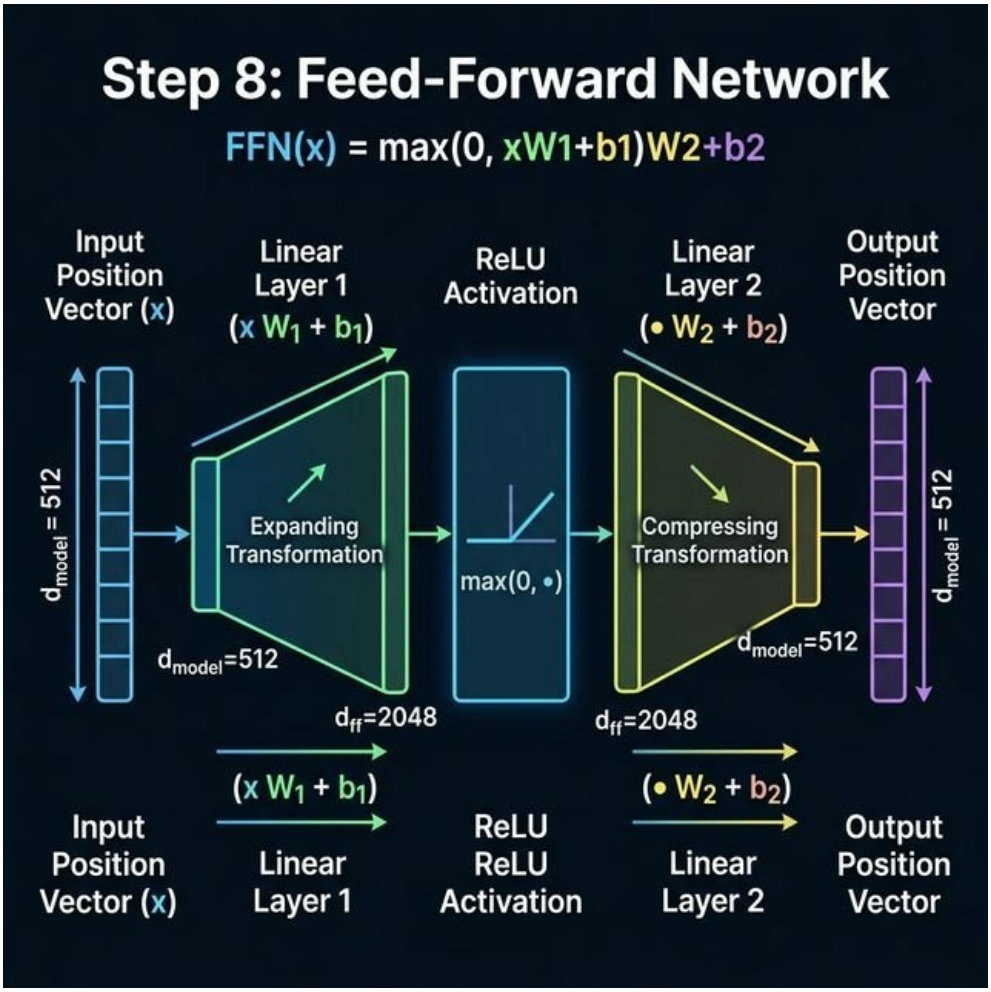
```
z = [4.0, 2.0, 6.0, 0.0]
μ = (4 + 2 + 6 + 0) / 4 = 3.0
σ = sqrt(((1)² + (-1)² + (3)² + (-3)²) / 4) = sqrt(5) ≈ 2.236

Normalized = [(4-3)/2.236, (2-3)/2.236, (6-3)/2.236, (0-3)/2.236]
           = [0.447, -0.447, 1.342, -1.342]
```

Why normalize? Each layer's output can drift in magnitude. Without normalization, values grow or shrink across layers, making training unstable.

> 🔍 **Why Layer Norm, not Batch Norm? Batch Normalization** (Ioffe & Szegedy, 2015) normalizes across the *batch dimension* — it needs statistics from multiple examples. In sequence tasks, batch sizes vary and sequences have variable lengths, making batch statistics unreliable. **Layer Normalization** (Ba et al., 2016, arXiv:1607.06450) normalizes across the *feature dimension* within a single example — no dependency on batch size.

STEP 8: Position-wise Feed-Forward Network (The "Thinking" Layer)



Step 8: Feed-Forward Network — expand, activate, compress

What Happens Here

After attention determines *which tokens are relevant*, the FFN (Feed-Forward Network) processes that information at each position independently:

```
FFN(x) = max(0, x · W₁ + b₁) · W₂ + b₂

Where:
  W₁: (512 × 2048)   ← EXPAND from 512 to 2048
  b₁: (2048)
  W₂: (2048 × 512)   ← COMPRESS back from 2048 to 512
  b₂: (512)
  max(0, ...) = ReLU activation (Rectified Linear Unit)
```

Step-by-Step Computation

```
Input x:         [0.21, -0.45, 0.87, ...]         ← 512 dimensions

Step 1: Expand
  z = x · W₁ + b₁                                  ← 2048 dimensions
  z = [1.23, -0.67, 2.45, -1.89, 0.34, ...]

Step 2: ReLU activation
  ReLU(z) = max(0, z)                              ← set negatives to zero
  ReLU(z) = [1.23, 0.00, 2.45, 0.00, 0.34, ...]
```

```
Step 3: Compress
  output = ReLU(z) · W₂ + b₂                        ← back to 512 dimensions
  output = [0.56, 0.12, -0.34, ...]
```

Why Expand Then Compress? (The Bottleneck Design)

```
512 ──► 2048 ──► 512

          ┌──────────────────┐
          │  "thinking space" │      ← 4× larger inner dimension
          └──────────────────┘         gives more room for complex
                                       feature transformations
          ┌──────────┐
          │ output │                   ← compressed back to original size
          └──────────┘
```

The expansion to 2048 allows the network to represent more complex transformations. The compression back to 512 keeps the model dimension consistent across layers.

> 🔍 **What if we skip the FFN entirely?** Attention only computes weighted averages — it's a **linear** operation over values. Without the FFN's nonlinearity (the ReLU activation), the Transformer would be a stack of linear transformations, which collapses into a single linear transformation. The FFN introduces **non-linearity**, giving the model the ability to learn complex functions. The paper's ablation shows this is critical for performance.

> 🔍 **Why "position-wise"?** The same FFN weights W_1, W_2 are applied at every position independently. Position 1 and position 100 use the same weights but different inputs. This is unlike a standard fully-connected layer that would mix all positions together.

Parameter Count for FFN

```
W₁: 512 × 2048 = 1,048,576
b₁: 2048       = 2,048
W₂: 2048 × 512 = 1,048,576
b₂: 512        = 512
                 ──────────────
Total:           2,099,712 parameters per layer (≈ 2M)
```

The FFN has **2× more parameters** than attention — it's where most of the model's "knowledge" is stored!

STEP 9: The Complete Encoder (Putting It All Together)

One **encoder layer** stacks Steps 4–8:

```
Input X (seq_len × 512)
  │
  ├──────────────────────────────────────────┐ (residual)
  │                                          │
  ▼                                          │
  Multi-Head Self-Attention (Step 4-6)       │
  │                                          │
```

```
  ▼                                          │
  Add (X + Attention_output) ◀──────────────┘
  │
  ▼
  Layer Norm
  │
  ├──────────────────────────────────────────┐ (residual)
  │                                          │
  ▼                                          │
  Feed-Forward Network (Step 8)              │
  │                                          │
  ▼                                          │
  Add (prev + FFN_output) ◀─────────────────┘
  │
  ▼
  Layer Norm
  │
  ▼
Output (seq_len × 512)
```

This entire block is repeated N = 6 times with different learned weights.

```
X → [Encoder Layer 1] → [Encoder Layer 2] → ... → [Encoder Layer 6] → Encoder Output
```

The final encoder output (shape: seq_len × 512) is a rich, contextualized (each token now carries information from ALL other tokens) representation of the input.

STEP 10: Decoder — Masked Self-Attention (No Peeking at the Future!)

The Problem

During training, we feed the entire target sequence to the decoder. But during generation, the model predicts one token at a time — it shouldn't see future tokens. If it could see the answer, it would just copy it instead of learning.

The Masking Matrix

```
Target: "J' adore apprendre"    (4 tokens)

Attention Mask (1 = can see, 0 = blocked):

            J'    adore  apprend  re
    J'    [ 1      0       0      0  ]  ← J' can only see itself
   adore  [ 1      1       0      0  ]  ← "adore" can see J' and itself
  apprend [ 1      1       1      0  ]  ← can see everything before it
    re    [ 1      1       1      1  ]  ← last token sees everything

Upper triangle = MASKED (set to -∞ before softmax)
```

How Masking Works Mathematically

```
Raw scores (Q · K^T / √d_k):

            J'    adore  apprend  re
```

```
    J'     [ 2.1    1.3     0.8    0.5 ]
   adore   [ 0.9    3.2     1.1    0.7 ]
  apprend  [ 0.4    0.6     2.8    1.9 ]
    re     [ 0.3    0.5     0.7    3.1 ]

After applying mask (add -∞ to upper triangle):

             J'    adore  apprend  re
    J'     [ 2.1    -∞      -∞     -∞  ]
   adore   [ 0.9    3.2     -∞     -∞  ]
  apprend  [ 0.4    0.6     2.8    -∞  ]
    re     [ 0.3    0.5     0.7    3.1 ]

After softmax (e^(-∞) = 0, so masked positions get zero weight):

             J'    adore  apprend  re
    J'     [ 1.00   0.00    0.00   0.00 ]  ← J' attends only to itself
   adore   [ 0.09   0.91    0.00   0.00 ]  ← adore attends to J' and itself
  apprend  [ 0.07   0.09    0.84   0.00 ]  ← apprend sees first 3 tokens
    re     [ 0.05   0.06    0.08   0.81 ]  ← re sees everything
```

> 🔍 **What if we don't mask future tokens in the decoder?** During training, the model would "cheat" — it would just copy the next token from the future. It would achieve perfect training loss but learn nothing. At inference time (actual generation), when future tokens don't exist, it would fail completely. This is called **data leakage** (information from the answer contaminating the learning process).

> 🔍 **Why not mask in the encoder too?** The encoder processes the **input** (source language), which is fully available. There's no "future" to hide — for understanding "I love learning," you need ALL words simultaneously. Only the **decoder** (generating output) needs masking because it must learn to predict without seeing the answer.

STEP 11: Cross-Attention (Decoder Reads the Encoder)

The Connection Between Encoder and Decoder

This is where the two halves of the model communicate:

```
Cross-Attention:
  Query (Q) ← from DECODER (what the decoder is looking for)
  Key   (K) ← from ENCODER (what the encoder offers)
  Value (V) ← from ENCODER (the encoder's information)

  Decoder token "adore" asks: "Which input tokens are relevant to me?"
  Answer: "love" has the highest attention score → gather its information
```

```
                    ENCODER OUTPUT
                    (seq_len_src × 512)
                         │
                    ┌────┴────┐
                    │  K   V  │     ← Keys and Values come from encoder
                    └────┬────┘
                         │
```

```
                        ┌─────┴─────┐
    DECODER    ──────▶  │     Q     │     ← Query comes from decoder
    hidden state        └───────────┘
                              │
                       Attention(Q_dec, K_enc, V_enc)
                              │
                        ┌─────┴─────┐
                        │  Output   │  ← Decoder position now has encoder info
                        └───────────┘
```

Concrete Example

```
Encoder input:  "I love learning"  → Encoder produces K_enc, V_enc
Decoder at position 2 ("adore"):

  Q_decoder = projection of decoder's hidden state for "adore"

  Attention scores (Q_decoder · K_encoder^T / √d_k):
    "I": 0.05,  "love": 0.85,  "learning": 0.10

  Output = 0.05·V("I") + 0.85·V("love") + 0.10·V("learning")
         ≈ V("love")   ← decoder gets the meaning of "love" from encoder
```

> 🔍 **What if we skip cross-attention (decoder doesn't look at encoder)?** The decoder would be a standalone language model with no access to the input. It would generate fluent text but with no relation to the input. For translation: it might output valid French, but not a translation of the English input.

> 🔍 **Why does Q come from the decoder but K, V from the encoder? Q = "What am I looking for?"** — this must come from the decoder's current state (what it needs to generate next). **K,V = "What's available?"** — this is the encoder's understanding of the input. The decoder is *querying* the encoder's representation.

STEP 12: The Complete Decoder Layer

One decoder layer has THREE sub-layers (vs encoder's two):

```
Target Input (shifted right)
  │
  ▼
  ┌──────────────────────────────────────┐
  │ 1. MASKED Self-Attention (Step 10)   │  ← decoder attends to itself
  │    + Add & Norm                      │     (with future masking)
  └──────────────────────────────────────┘
                  │
                  │
  ┌──────────────────────────────────────┐
  │ 2. CROSS-Attention (Step 11)         │  ← decoder attends to encoder
  │    Q from decoder, K/V from encoder  │
  │    + Add & Norm                      │
  └──────────────────────────────────────┘
                  │
                  │
  ┌──────────────────────────────────────┐
  │ 3. Feed-Forward Network (Step 8)     │  ← same structure as encoder FFN
  │    + Add & Norm                      │
```

```
  └──────────────┬──────────────────────────┘
                 │
                 ▼
          Decoder Layer Output
```

Repeated **N = 6 times** with different weights.

STEP 13: Linear Projection + Softmax (Producing the Final Output)

The Final Layers

After 6 decoder layers, we have a vector of shape (seq_len × 512) for each decoder position. To predict the next token:

```
Step 1: Linear Projection
  logits = decoder_output · W_linear + b_linear

  Shape: (seq_len × 512) · (512 × vocab_size) = (seq_len × 37000)

  Each position now has a score for EVERY word in the vocabulary.

Step 2: Softmax (Convert to Probabilities)
  probabilities = softmax(logits)     ← each row sums to 1.0

Example for position 2:
  logits:        [... -2.3, 1.1, 5.8, 0.4, 3.2, ...]   (37,000 values)
  After softmax: [... 0.00, 0.01, 0.72, 0.00, 0.05, ...] (37,000 probabilities)
                                      ────
                                      highest probability → "adore" (token ID 5834)
```

Weight Tying (Parameter Sharing Trick)

The original Transformer paper uses **tied weights**: the linear projection matrix W_linear is the **transpose of the input embedding matrix E^T**.

```
W_linear = E^T    (shape: 512 × vocab_size)

Why?
  - Saves ~19M parameters (512 × 37000 = 18.9M)
  - Makes intuitive sense: if embedding maps word→vector,
    the reverse mapping (vector→word) should use the same relationship
```

(Reference: Press & Wolf, 2017, “Using the Output Embedding to Improve Language Models,” arXiv:1608.05859)

> 🔍 **What if we use argmax instead of softmax during training?** Argmax is not differentiable (you can’t compute gradients through it). Training would be impossible. Softmax provides smooth probabilities that enable gradient-based learning. During **inference** (actual generation), we can use argmax (greedy decoding) or sampling.

STEP 14: Training — The Loss Function

Cross-Entropy Loss

```
For each position, the model predicts a probability distribution over the vocabulary.
We compare this to the true next token using Cross-Entropy (CE) Loss:

  L = -Σ y_true · log(y_predicted)

Since y_true is one-hot (only 1 for the correct token):
  L = -log(P(correct_token))

Example:
  True next token: "adore" (ID: 5834)
  Model's probability for "adore": 0.72

  Loss = -log(0.72) = 0.329

  If P("adore") = 0.01 → Loss = -log(0.01) = 4.605  ← HIGH loss, bad prediction
  If P("adore") = 0.99 → Loss = -log(0.99) = 0.010  ← LOW loss, correct!
```

Teacher Forcing (How Training Works)

During training, the decoder receives the **correct previous tokens** as input (not its own predictions). This is called **teacher forcing**:

```
Training:
  Decoder input:    [START,  "J'",  "adore",  "apprendre"]
  Expected output:  ["J'",  "adore", "apprendre",  END]

  Teacher forcing: always feed the CORRECT previous token,
  even if the model predicted wrong.

Inference (generation):
  Step 1: Input [START]          → Predict "J'"
  Step 2: Input [START, "J'"]    → Predict "adore"
  Step 3: Input [START, "J'", "adore"] → Predict "apprendre"
  ...continue until END token is predicted
```

> 🔍 **What if we don't use teacher forcing?** If the model makes an error on token 2, that error propagates — token 3's input is wrong, making token 3's prediction worse, and so on. This is called **exposure bias** — the gap between training (always correct input) and inference (potentially wrong input). Techniques like scheduled sampling partially address this.

Complete Parameter Count (Original Transformer Base)

Component	Per Layer	× Layers	Total
Encoder Self-Attention	$4 \times (512 \times 64) \times 8 + 512^2$ = ~1.05M	×6	6.3M
Encoder FFN	512×2048 + 2048 + 2048×512 + 512 = ~2.1M	×6	12.6M
Decoder Masked Self-Attention	~1.05M	×6	6.3M
Decoder Cross-Attention	~1.05M	×6	6.3M

Decoder FFN	~2.1M	×6	12.6M
Input Embedding	37K × 512	×1	18.9M
Output Embedding (tied)	0 (shared with input)	—	0
Positional Encoding	0 (not learned)	—	0
			~63M total

This matches the paper's reported ~65M parameters for the base model (Section 3.3, Table 3).

Complete Data Flow — End to End

```
INPUT: "I love learning"

1. TOKENIZE          →  [34, 892, 1205, 418]
2. EMBED             →  shape: (4 × 512)
3. + POSITIONAL ENC  →  shape: (4 × 512)

4-8. ENCODER (×6 layers):
   ┌→ Multi-Head Self-Attention
   │    Q, K, V all from input
   │    8 heads, each d_k = 64
   ├→ Add & Norm
   ├→ Feed-Forward (512 → 2048 → 512)
   └→ Add & Norm

   Output: (4 × 512)  [rich contextual representation]

9-12. DECODER (×6 layers, autoregressive):
   ┌→ Masked Multi-Head Self-Attention
   │    Q, K, V from decoder input (future masked)
   ├→ Add & Norm
   ├→ Cross-Attention
   │    Q from decoder, K/V from encoder
   ├→ Add & Norm
   ├→ Feed-Forward (512 → 2048 → 512)
   └→ Add & Norm

13. LINEAR + SOFTMAX  →  probability over 37K vocab
14. PICK TOP TOKEN    →  "J'"
15. REPEAT from step 9 with new token appended

OUTPUT: "J' adore apprendre" [END]
```

Summary: Why Each Component Exists

Component	What It Does	What Breaks Without It
Tokenization	Splits text into subword units	Can't process raw strings; vocab too large/small
Embedding	Gives tokens dense semantic meaning	No way to represent word similarity
Positional Encoding	Encodes word order	"Dog bites man" = "Man bites dog"
Q, K, V projections	Create role-specific representations	Can't learn asymmetric attention patterns

Scaled Dot-Product	Measures relevance between tokens	Without scaling: softmax saturates, gradients die
Multi-Head	Captures multiple relationship types simultaneously	Single relationship per position
Residual connections	Preserves information across layers	Gradients vanish in deep networks
Layer Normalization	Stabilizes activations	Training becomes unstable, values diverge
FFN	Adds non-linearity and “thinking”	Model becomes purely linear (collapses)
Decoder Masking	Prevents seeing future tokens	Model cheats during training
Cross-Attention	Lets decoder read the encoder’s output	Output ignores input completely
Softmax output	Produces probability distribution	Can’t train with gradients (not differentiable)

Key Reference

All architecture details, hyperparameters, and design decisions referenced in this document are from:

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). “Attention Is All You Need.” *Advances in Neural Information Processing Systems, 30 (NeurIPS 2017)*. arXiv:1706.03762.

Additional cited works: - Sennrich et al. (2016). “Neural Machine Translation of Rare Words with Subword Units.” arXiv:1508.07909 — BPE tokenization - He et al. (2015). “Deep Residual Learning for Image Recognition.” arXiv:1512.03385 — Residual connections - Ba et al. (2016). “Layer Normalization.” arXiv:1607.06450 — Layer normalization - Press & Wolf (2017). “Using the Output Embedding to Improve Language Models.” arXiv:1608.05859 — Weight tying - Su et al. (2021). “RoFormer: Enhanced Transformer with Rotary Position Embedding.” arXiv:2104.09864 — RoPE